

Embedded Systems (8-bit, 16-bit & 32-bit Controllers) – Comprehensive Syllabus

Course Overview

This course provides a comprehensive understanding of embedded systems using 8-bit, 16-bit, and 32-bit microcontrollers. Students will learn embedded C programming, microcontroller architecture, peripheral interfacing, communication protocols, RTOS fundamentals, and real-time embedded application development through hands-on projects.

Duration: 16 Weeks (80–100 Hours)

Prerequisites:

- Basic knowledge of C Programming
 - Digital Electronics fundamentals
 - Basic Electrical & Electronics concepts
-

Module 1: Introduction to Embedded Systems (Week 1)

Topics

- Introduction to Embedded Systems
- Embedded System Architecture
- Hardware vs Software
- Types of Embedded Systems
- Real-Time Systems
- Embedded Development Process
- Applications of Embedded Systems

Practical Exercises

- Development Environment Setup
 - IDE Installation
 - Introduction to Development Boards
-

Module 2: C Programming for Embedded Systems (Week 2)

Topics

- Data Types
- Variables
- Operators
- Control Statements
- Functions
- Arrays
- Strings
- Structures
- Unions
- Bit Manipulation
- Pointers
- Memory Management

Practical Exercises

- Embedded C Programs
 - Bitwise Operations
 - Register-Level Programming
-

Module 3: 8-bit Microcontroller Architecture (8051/PIC/AVR) (Week 3)

Topics

- Introduction to 8-bit Controllers
- CPU Architecture
- Memory Organization
- Registers
- Stack
- Program Counter
- Special Function Registers (SFR)
- Clock System

Practical Exercises

- LED Blinking

- Switch Interfacing
 - Delay Generation
-

Module 4: GPIO & Peripheral Interfacing (Week 4)

Topics

- GPIO Configuration
- Digital Input
- Digital Output
- Push Buttons
- LEDs
- Relays
- Buzzer
- LCD (16x2)
- Seven Segment Display
- Matrix Keypad

Practical Exercises

- LED Patterns
 - LCD Display
 - Keypad Password System
-

Module 5: Timers, Counters & Interrupts (Week 5)

Topics

- Timers
- Counters
- Interrupts
- External Interrupts
- Interrupt Service Routine (ISR)
- PWM Basics

Practical Exercises

- Timer Delay
 - PWM LED Brightness
 - External Interrupt Demo
-

Module 6: Serial Communication Protocols (Week 6)

Topics

- UART
- SPI
- I2C
- RS232
- RS485
- CAN Basics

Practical Exercises

- UART Communication
 - LCD via I2C
 - SPI Sensor Interface
-

Module 7: Sensors & Actuators (Week 7)

Topics

- Temperature Sensors
- Ultrasonic Sensors
- IR Sensors
- LDR
- Gas Sensors
- Humidity Sensors
- DC Motor
- Servo Motor
- Stepper Motor
- Relay Control

Practical Exercises

- Temperature Monitoring

- Distance Measurement
 - Motor Speed Control
-

Module 8: ADC & DAC (Week 8)

Topics

- Analog vs Digital Signals
- ADC Working
- DAC Basics
- Sensor Data Acquisition
- Signal Conditioning

Practical Exercises

- Potentiometer Reading
 - Temperature Sensor Interface
 - Analog Voltage Measurement
-

Module 9: 16-bit Microcontrollers (MSP430/PIC24) (Week 9)

Topics

- Introduction to 16-bit Controllers
- Architecture
- Clock System
- Low Power Modes
- Memory Organization
- GPIO
- Timers

Practical Exercises

- LED Control
 - Timer Applications
 - UART Communication
-

Module 10: 32-bit ARM Cortex-M Microcontrollers (STM32) (Week 10)

Topics

- ARM Architecture
- Cortex-M Series
- STM32 Family
- Registers
- Memory Map
- GPIO
- RCC
- NVIC
- Clock Configuration

Practical Exercises

- STM32CubeIDE
 - LED Blinking
 - Push Button Interface
-

Module 11: Advanced Peripheral Programming (Week 11)

Topics

- UART
- SPI
- I2C
- ADC
- DAC
- PWM
- DMA
- Watchdog Timer

Practical Exercises

- Multi-Sensor Interface
- PWM Motor Control
- DMA Data Transfer

Module 12: RTOS Fundamentals (Week 12)

Topics

- Introduction to RTOS
- Task Scheduling
- Threads
- Semaphores
- Mutex
- Queues
- Event Flags
- FreeRTOS Basics

Practical Exercises

- LED Task Scheduling
 - Producer-Consumer Example
 - Queue Communication
-

Module 13: Embedded Communication Protocols (Week 13)

Topics

- CAN Protocol
- LIN Protocol
- Modbus
- Ethernet Basics
- USB Basics
- Bluetooth
- Wi-Fi Basics

Practical Exercises

- CAN Communication
- Bluetooth Interface
- IoT Sensor Node

Module 14: Embedded Linux & IoT Basics (Week 14)

Topics

- Introduction to Embedded Linux
- Raspberry Pi Basics
- Linux Commands
- Python for Embedded Applications
- MQTT
- IoT Architecture
- Cloud Integration

Practical Exercises

- Raspberry Pi GPIO
 - MQTT Communication
 - IoT Dashboard
-

Module 15: Mini Project (Week 15)

Choose one:

- Smart Home Automation
 - Digital Temperature Controller
 - Smart Parking System
 - Automatic Street Light
 - Water Level Monitoring System
 - Smart Irrigation System
 - RFID Attendance System
-

Module 16: Capstone Project (Week 16)

Complete Embedded Product Development

- Requirement Analysis

- Circuit Design
 - Component Selection
 - Firmware Development
 - Sensor Integration
 - Communication Protocol Implementation
 - Testing & Debugging
 - PCB Basics
 - Documentation
 - Product Demonstration
-

Tools & Software

- Embedded C
 - Keil uVision
 - MPLAB X IDE
 - STM32CubeIDE
 - Code Composer Studio (CCS)
 - Proteus
 - Arduino IDE
 - STM32CubeMX
 - Git & GitHub
 - Serial Terminal Tools (PuTTY/Tera Term)
-

Hardware Platforms

- **8-bit:** 8051, AVR (ATmega328P), PIC16F877A
 - **16-bit:** MSP430, PIC24
 - **32-bit:** ARM Cortex-M (STM32F4/F1), LPC1768
 - Arduino Uno/Nano
 - STM32 Nucleo/Discovery Boards
 - Raspberry Pi (for Embedded Linux & IoT)
-

Assessment Scheme

Component	Weightage
-----------	-----------

Assignments	15%
Lab Exercises	25%
Quizzes	10%
Mini Project	20%
Capstone Project	30%

Learning Outcomes

By the end of the course, learners will be able to:

- Understand the architecture and operation of 8-bit, 16-bit, and 32-bit microcontrollers.
- Develop embedded applications using Embedded C.
- Interface sensors, actuators, displays, and communication peripherals.
- Configure and use timers, interrupts, ADC, DAC, PWM, and GPIO.
- Implement serial communication protocols including UART, SPI, I²C, CAN, and Bluetooth.
- Develop multitasking applications using FreeRTOS.
- Design and debug embedded systems using industry-standard development tools.
- Build end-to-end embedded solutions integrating hardware, firmware, and communication interfaces.
- Develop real-world embedded and IoT projects suitable for automotive, industrial automation, consumer electronics, and smart device applications.